Mutation Testing
& Test Effectiveness

If this line were wrong, would your tests complain?

SYSTEM UNDER TEST
e-Shop backend

TOOLING
StrykerJS 9.6.1 + Jest 30

apps/backend/server.js

```
if (total_amount > coupon.min_order_amount)
                 | if this were >= …
```

This technique grades the **test suite**, not the production code. Every number in this talk is measured, not illustrative.

AGENDA | 02

What we will go through

One continuous argument in six stages — the problem first, each term introduced only where the argument needs it.

01 **WHY**
High coverage can still mean a weak test suite

02 **WHAT**
Mutants, mutation score, and the RIPR model

03 **WHERE**
Where it pays off — and where it stops being cheap

04 **HOW**
The 7-step loop and CI/CD integration

05 **AI**
Who grades agent-written tests — and how AI helps back

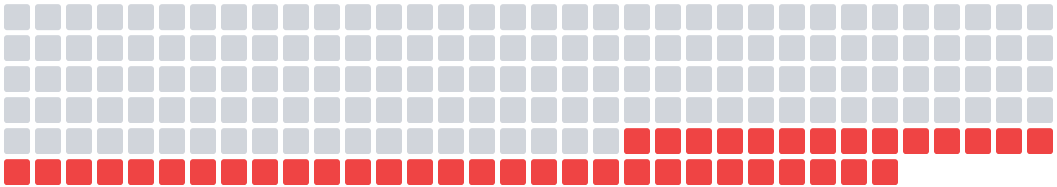
06 **DEMO**
e-Shop: one surviving mutant, fixed, re-run

40/40 tests pass. 97% line coverage. So is the suite good?

e-Shop backend · the 4 routes our group committed to test (FR-02, FR-08, FR-09, FR-10). Every number here is measured on those **same four routes**.

NPM TEST	LINE COVERAGE	BRANCH COVERAGE
40/40	97% 74/76	90% 52/58

We then generated **215 single-line wrong versions** of that same code — one changed operator, one changed constant, one changed line each — and re-ran all 40 tests against every one of them.



156 — a test failed, so it was caught

43 — all 40 tests still reported green

215 = 199 + 16

199 run inside code the tests **do** execute. Only 16 are never reached — so this is **not a coverage problem**.

MUTATION SCORE
78.39%

ROUTE	LINE	BRANCH
FR-02 · POST /api/login	100%	92%
FR-09 · POST /api/apply-coupon	93%	85%
FR-08 · POST /api/cart + /api/checkout	100%	75%
FR-10 · PUT /api/admin/orders/:id/status	100%	95%

Coverage measures execution. It does not measure verification.

THIS TEST REACHES 100% COVERAGE ON `createOrder()`

```
it('creates order', async () => {
  expect(await service.createOrder(req)).not.toBeNull();
});
```

EXECUTED — 5/5 LINES

```
map
calculateTotal
save
publish(OrderCreated)
```

VERIFIED

None of them

Delete the `publish` line, or change the total formula — this test stays green.

DEFINITION · ORACLE

Whatever a test uses to decide pass or fail.

Here the oracle is one thing only: “not null”. Five lines executed, one trivial property checked.

DEFINITION · INCIDENTAL COVERAGE

Code executed as a side effect of another test, while its behaviour is not part of that test’s oracle.

COVERAGE ASKS

Was this line executed?

MUTATION TESTING ASKS

If this line were wrong, would a test fail?

Inject a fake fault into the code, then see if the tests complain

ORIGINAL

```
if (expiry < now)
```

MUTANT · EqualityOperator

```
if (expiry <= now)
```

Mutants never reach production: generated in an isolated sandbox, executed, thrown away.

Run only the tests that execute this line

test FAIL
KILLED
The suite detected the fault — good.

test PASS
SURVIVED
The suite is blind to it — bad.

KEY IDEA

A mutant is a controlled fault hypothesis, not a random bug.

"If a developer wrote <= instead of <, would the current suite catch it?"

ASSUMPTION

Competent programmer hypothesis

Real faults are usually small deviations from the correct program, not a rewritten function.

Five states — and the one distinction that changes your fix

STATE	CONDITION	WHAT IT MEANS
KILLED	at least 1 test fails	detected
SURVIVED	tests DID run, none failed	**Oracle gap** → stronger assertion, or better data
NO COVERAGE	no test touches it	**Coverage gap** → a new test case
TIMEOUT	exceeds the time threshold	counted as detected
INVALID	does not compile / crashes	excluded from the denominator

DO NOT MERGE THESE TWO

Survived and No Coverage both read as “not detected”, so dashboards merge them.

Same symptom, opposite fix. Merge them and you throw away the diagnosis.

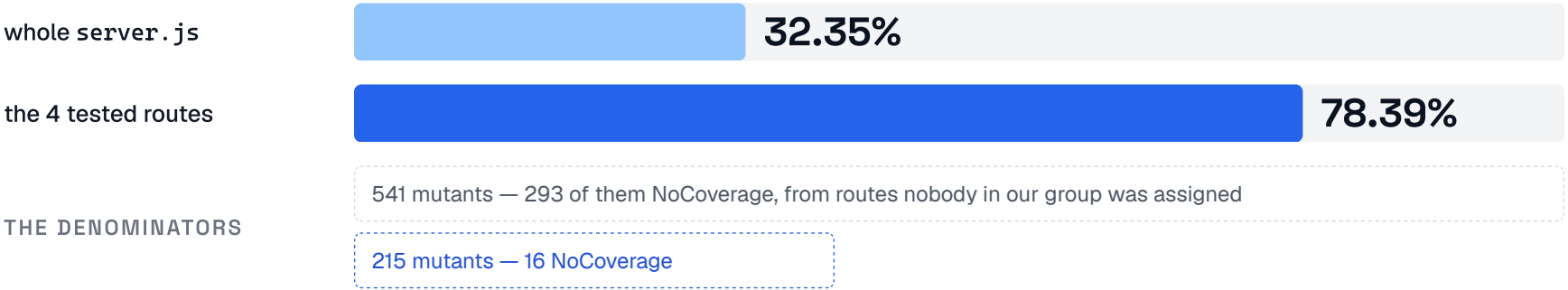
NOT A SIXTH STATE

EQUIVALENT is a manual label on a subset of Survived rows — it comes back on slide 10a.

Same suite, same run — two different scores

MUTATION SCORE
`Killed / (Total − Equivalent − Invalid)`

TEST STRENGTH
`Killed / (Killed + Survived)` — excludes NoCoverage



Same suite. Same run. Same day. Only the denominator changed.

Never compare two mutation scores unless tool, version, operator set, file scope, exclusion rules and formula all match. **Six conditions, not one.**

RIPR: why did this mutant survive?

Reachability
Does a test reach that line?

→

Infection
Does the state actually change?

→

Propagation
Does the difference reach an observable output?

→

Revealability
Is the assertion strict enough to tell them apart?

WHAT BROKE	THE FIX	FIXABLE BY A TEST?
A RIPR LINK BROKE		
Reachability	add a test case / precondition	YES
Infection	change test data — a=b=0 makes a+b and a−b identical	YES
Propagation (a) — dead or redundant code	fix the production code	NO
Propagation (b) — masked by a clamp, rounding, validation	move the test data out of the masked range	YES
Revealability	a precise assertion — assertEquals, not result >= 0	YES
NOT A RIPR BREAK — "WRITE MORE TESTS" IS THE WRONG REFLEX		
Equivalent mutant	cannot be killed — document the reason, suppress narrowly	NO
Ambiguous requirement	stop — ask the BA before touching the test	NO

Put it at the lowest level where the behaviour is still observable

LEVEL	WHAT IT TARGETS	POLICY
API / E2E	public contracts, authorization, idempotency	selective
Integration	transactions, locking, messaging, serialization	selective
Component	workflows, state transitions, failure paths	selective
Unit	business rules, calculations, validation, boundaries	DEFAULT

e-SHOP · FR-09 POST /api/apply-coupon · UNIT

5 sequential guards + 2 calculation branches, in one endpoint.

81
mutants in a single route

69.3%
kill rate — lowest of our four

High logic density is exactly where this technique pays.

WHY THE LOWEST LEVEL

Fewer intermediate systems → the RIPR chain completes more easily → clean signal, one fault hypothesis per survivor.

BLIND SPOT

Over-mocking hides real integration behaviour — side effects get verified as mock interactions, not real results.

EXCEPTION · MONEY, PERMISSIONS, COMPLIANCE, TENANT ISOLATION

Defence in depth across levels. One survivor in authorization outweighs hundreds in formatting code.

Where this technique stops being cheap

01 EQUIVALENT MUTANT

Cannot be killed. Cannot be detected automatically. Reported rates 4%–39%, and every one of them distorts the denominator.

The biggest one — it gets the next two slides.

02

COST

```
≈ N_mutants × N_tests_per_mutant
× time_per_test
```

Without test selection this is a product, not a sum.

8 min 30 s

e-Shop: 541 mutants, 40 tests, concurrency 1

03

FLAKY TESTS

False kills, fake timeouts, scores that jump between runs.

A mutant killed only by a flaky test is not a clean kill — mark it indeterminate.

04

SMALL SAMPLES

2 mutants, 1 killed = 50%. One mutant moves the score by 50 points.

Fewer than 10 mutants in the diff: review them one by one, do not gate on a percentage.

Equivalent mutants: when “survived” is not the test’s fault

DEFINITION · EQUIVALENT MUTANT

1. **Syntax differs** from the original — the source really changed
2. **Behaviour is identical** for every valid input
3. ⇒ **no test case, however strong, can kill it**

“Valid” = the input domain the real system can actually produce. Who decides that domain? A **human**, not the tool.

WHY NO TOOL WILL DO THIS FOR YOU

Detecting it reduces to program equivalence — undecidable.

In the report it looks exactly like a weak test. Both carry the same label:

SURVIVED

THE OBSERVATION THAT MAKES THE NEXT SLIDE READABLE

A relational-operator mutant differs from the original at exactly one point: where both sides are equal. Everywhere else they agree.

So the whole question collapses to one: can the program reach that point?

Same transformation, three verdicts

Can the program reach the divergence point?

KILLED #368 · line 391

`usage_count >= max → > max`

DIVERGENCE

`usage_count == max`

Reachable, and tested: TC-BVA-05/06 sit exactly there (uses=1,max=1 · uses=2,max=2).

SURVIVED — REAL GAP #355 · line 382

`expiry < now → <= now`

DIVERGENCE

`expiry == now, to the millisecond`

11 tests execute this line; none stands on that point. Freeze the clock and it is killable — Demo A.

EQUIVALENT #46 · line 56

`newAttempts >= 3 → > 3`

DIVERGENCE

`newAttempts == 3`

login_attempts goes 0 → 2 → 4 … (step +2). newAttempts is always even. 3 never occurs.

CONTROL PAIR
line 56 · EqualityOperator

#46	>= 3 → > 3	SURVIVED
#47	>= 3 → < 3	KILLED

Same line, same four covering tests — one dies, one survives. So “Survived” on #46 does not mean this line needs more tests.

CAVEAT 1

Seed `login_attempts = 1` straight into SQLite and #46 becomes killable — so the label is “equivalent **candidate**, awaiting sign-off”.

CAVEAT 2

Dataflow says “equivalent”. Only the spec says whether locking on the 2nd failure is a bug.

The seven-step loop

A HUMAN DECIDES THE TOOL DOES THIS

01 · GATE

Green baseline

Tests must pass 100% first. Not green → stop.

Otherwise you cannot tell a mutant failure from a pre-existing one.

THE TOOL DOES THIS — StrykerJS, PIT, MutPy all automate steps 2–5

02

Generate mutants

AST or bytecode; mutation switching — instrument once, toggle each mutant

03

Dry run

Run the suite once: confirm green, record test-to-mutant coverage, set the timing baseline

04

Selective run

Per mutant, run only the tests that cover it — naive N × M does not scale

05

Classify

Killed / Survived / NoCoverage / Timeout / Invalid

06 · WHERE VALUE IS CREATED

Fix

tighten an assertion
change the test data
add a test case
refactor the production code
clarify the requirement
record an equivalent

Six directions, picked with the RIPR diagnosis.

07

Re-run and confirm

The target mutant moved to Killed.

07 → 04 Loop back to the selective run — no need to regenerate mutants.

Into CI/CD: cheap enough to run, gated on risk

FOUR COST LEVERS

1 · Fewer mutants
diff scope, module rotation, a stable operator set

2 · Fewer tests per mutant
coverage-based selection, bail on first kill

3 · Cheaper tests
remove sleeps, inject the clock, deterministic fakes

4 · Incremental
reuse previous results, run only what the diff affects

Reuse needs precise change detection: Jest matches per test, Vitest and Mocha per file — touch one test and every mutant that file covers re-runs. Your runner choice caps this lever.

TIER	TRIGGER	SCOPE
0	before commit	1 file / 1 class, local
1	every PR	changed code, unit tests, incremental
2	PR touches payment / auth / tenant	+ component + selected integration
3	nightly	module rotation, refresh the baseline
4	before release	critical services + manual survivor review

Full-repo cost grows with the size of the codebase. Diff-scoped cost is bounded by the size of the change.

More workers only buy time on the first one.

Not one repo-wide percentage — that teaches the team to optimise the *number*, not the risk.

BLOCK

survivor on authorization or money logic · critical code changed with NoCoverage · a new survivor nobody can explain

WARN

small score drop · equivalent candidates awaiting review · fewer than 10 mutants in the diff

AI-written tests: the structure is there, the oracle is not

CURRENT PRACTICE

agent writes tests — tests pass — coverage rises — merge

That is the entire acceptance criteria.

Exactly the criteria slides 3–4 proved insufficient — now applied to hundreds of tests generated in seconds.

80.2%

of agent-authored test patches have a weak oracle, or no explicit oracle at all

There *is* a test file. There *is* a test function. The code *is* called. Nothing verifies whether the behaviour is right.

THREE WAYS THE ORACLE BREAKS — ALL THREE LOOK REASONABLE WHEN YOU READ THE CODE

01
SELF-CONFIRMING
Asserts back the value the implementation just returned.

02
IMPLEMENTATION-BOUND
Asserts iteration order or internal structure.

03
WRONG ORACLE
Recalls buggy logic from training data and asserts it is correct.

Only one way to know: give the suite a known behavioural deviation and see if it catches it — mutation testing.

Mutation-feedback prompting: the gate is the command you just ran

WEAK PROMPT

“Write tests for this function.”

No correctness criterion. Not checkable.

STRONG PROMPT

“Write a test that PASSES on the original and FAILS on this mutant.”

Binary criterion. Machine-checkable. The only difference: it ships a concrete deviation with it.

CALLBACK · SLIDE 13

The LLM’s assertion fails in the same three ways — self-confirming · implementation-bound · wrong oracle. None visible by reading. Only the re-run separates them.

THE SAME SEVEN-STEP LOOP, AI INSERTED AT THREE POINTS

1–4 · CLI	generate · run · classify — deterministic, AI does not touch this
5 · LLM	read the survivor — which RIPR link is broken?
6 · LLM + HUMAN	suspected equivalent → LLM triages, a human signs. Otherwise → LLM writes an assertion targeting the mutant.
7 · CLI	**re-run — this is the Validation Gate**

Re-run the exact command from step 1. Open the report, find that mutant ID: did **Survived** become **Killed**?

BLOCK

Did not flip — the assertion is useless. Reject it.

WARN

Flipped, but the oracle is not business-correct — human review blocks it (HG-3).

Where AI plugs in — and where the line is

DETERMINISTIC CORE `generate mutants → selective run → classify → compute score` — AI does not touch this — and that is a good thing.

INSERTION POINT	USABLE NOW?	MAIN RISK
1 · Fix tests write an assertion targeting a survivor	YES — chat + CLI	must pass the Validation Gate
2 · Screen equivalents LLM triages equivalent candidates	YES — LOW CONFIDENCE	false negative → the score improves artificially
3 · Generate / prioritise mutants LLM proposes realistic-bug mutants · rank by kill history	NO	needs a plugin, or a kill-history database

WHAT AI DOES NOT REMOVE

- the equivalent mutant problem (undecidable)
- flaky tests
- the cost of running mutants
- the oracle problem
- the need for a human to sign off

THE BUTTON THAT ALREADY EXISTS

```
// Stryker disable next-line ConditionalExpression: <reason> — signed: <name>
```

The mutant becomes “Ignored” and leaves the denominator. Missing reason or missing name: no merge.

AI shortens triage time. It does not shorten responsibility.

Demo: one surviving mutant, audited and fixed by AI

SYSTEM UNDER TEST

POST /api/apply-coupon
FR-09 · server.js:363–443

5 sequential guards: is_active · min_order · expired_at · user_id · max_uses

2 calculation branches: percent / fixed

TWO TOOLS, NOTHING ELSE

StrykerJS 9.6.1
already installed, unchanged all talk

Claude Code
chat / agent CLI, no extra infrastructure

THE BASELINE THAT LOOKS FINE

```
$ npm test
40/40 pass
```

```
$ npm run test:coverage
server.js: 51.66% lines, 45.8% branches
```

```
$ npx stryker run --mutate "server.js:363-443"
81 mutants · 52 killed · 23 survived · 6 no-coverage
kill rate 69.3% — the lowest of our four routes
```

COMING UP — LIVE IN THE TERMINAL, NO SLIDE

01 · CLAUDE CODE
Audits one surviving mutant — diagnoses it with RIPR

→

02 · CLAUDE CODE
Writes an assertion targeting that exact mutant

→

03 · VALIDATION GATE
The same Stryker command above, re-run — it decides whether this merges